

# ASSIGNMENT – 7

## Simulation of TCP Flow Control Using SimPy

### AIM

To simulate TCP flow control using SimPy. A sender generates data segments and transmits them over a sliding window, but it is allowed to send only as much outstanding (unacknowledged) data as the receiver advertises. The receiver keeps a receive buffer of fixed size, reads segments out of it at a fixed processing rate, and advertises the free buffer space in every ACK and window update. When the advertised window becomes zero, the sender stops and waits for a window update before resuming.

The TCP Client module generates segments at a configurable sender rate, keeps track of the outstanding segments, and transmits a segment only when the outstanding data is below the effective window, which is the smaller of the initial window and the receiver's latest advertised window. While the window is full the sender waits, and each such wait is counted as a waiting event. The TCP Server module receives segments into a buffer, prints the buffer occupancy and advertised window for each segment, sends an ACK carrying the current advertised window, processes the oldest segment at the receiver rate, and then sends a window update with the new free space so a blocked sender can resume. Five scenarios are simulated with different segment sizes, initial windows, buffer sizes, and sender/receiver rates, and the throughput, waiting events, buffer occupancy, and full-buffer events are compared.

### ALGORITHM

- CLIENT: generate a segment at the sender rate and transmit it when the outstanding data is below the effective window.
- CLIENT: when the window is full, wait for an ACK or a window update from the receiver and count a waiting event.
- CLIENT: track segments sent, ACKs received, waiting events, transmission time, and throughput.
- SERVER: receive each segment into the receive buffer, print the buffer occupancy and advertised window, and send an ACK with the current advertised window.
- SERVER: process the oldest segment at the receiver rate and send a window update advertising the new free space.
- SERVER: detect full-buffer events and compute average and maximum buffer occupancy and the average advertised window.

### TCP CLIENT MODULE – tcp\_client.py

```
# tcp client simulation module: generates segments and transmits them only
# when space is available in the receiver's advertised window

class TCPClient:
```

```
def __init__(self, env, data_ch, ack_ch, total_kb, seg_kb, init_window_kb,
              sender_rate, delay, buffer_kb, out, detail):
    self.env = env
    self.data_ch = data_ch
    self.ack_ch = ack_ch
    self.total_kb = total_kb
    self.seg_kb = seg_kb
    self.init_window_kb = init_window_kb
    self.sender_rate = sender_rate
    self.delay = delay
    self.buffer_kb = buffer_kb
    self.out = out
    self.detail = detail
    self.transmitted = 0
    self.acks = 0
    self.waiting_events = 0
    self.outstanding = 0          # unacknowledged segments
    self.last_rwnd = buffer_kb    # receiver advertised window from acks

def handle_ack(self, msg):
    kind, seq, rwnd = msg
    self.last_rwnd = rwnd
    if kind == "ACK":
        self.outstanding -= 1
        self.acks += 1
        if self.detail:
            print(f"ACK Received : S{seq}", file=self.out)
            print(f"Advertised Window : {rwnd} KB", file=self.out)

def run(self):
    o = self.out
    print("TCP CLIENT SIMULATION", file=o)
    print(f"Total Data      : {self.total_kb} KB", file=o)
    print(f"Segment Size     : {self.seg_kb} KB", file=o)
    print(f"Initial Window     : {self.init_window_kb} KB", file=o)
    print(f"Sender Rate        : {self.sender_rate} KB/s", file=o)
    print(f"Transmission Delay: {self.delay} s", file=o)
    print(file=o)
    for i in range(1, self.total_kb // self.seg_kb + 1):
        # generate the next segment at the sender's rate
        yield self.env.timeout(self.seg_kb / self.sender_rate)
        # transmit only when the advertised window has space
        waited = False
        while self.outstanding * self.seg_kb >= min(self.init_window_kb, self.last_rwnd):
            # one waiting event per time the sender is held back
            if not waited:
```

```
        waited = True
        self.waiting_events += 1
        if self.detail:
            print("Sender Waiting - Receiver Window Full", file=o)
        # wait for an ack or a window update from the receiver
        ack = yield self.ack_ch.get()
        self.handle_ack(ack)
        if self.detail:
            print(f"Segment S{i} Transmitted", file=o)
        self.transmitted += 1
        self.outstanding += 1
        yield self.env.timeout(self.delay)
        yield self.data_ch.put((i, self.seg_kb))
    # drain the remaining acks until everything is acknowledged
    while self.outstanding > 0:
        ack = yield self.ack_ch.get()
        self.handle_ack(ack)
    print("Transmission Completed", file=o)
    print(f"Total Segments Sent      : {self.transmitted}", file=o)
    print(f"Total ACKs Received       : {self.acks}", file=o)
    print(f"Total Waiting Events       : {self.waiting_events}", file=o)
    print(f"Total Transmission Time: {self.env.now:.2f} s", file=o)
    print(f"Throughput                  : {self.total_kb / self.env.now:.2f} KB/s", file=o)
    print(file=o)
```

## TCP SERVER MODULE - tcp\_server.py

```
# tcp server simulation module: receives segments into a receive buffer,
# generates cumulative acks, processes them at the receiver's rate and
# advertises the available window (rwnd). receive and processing run as
# separate processes so the buffer can fill up while data is being processed

class TCPServer:
    def __init__(self, env, data_ch, ack_ch, buffer_kb, recv_rate, out, detail):
        self.env = env
        self.data_ch = data_ch
        self.ack_ch = ack_ch
        self.buffer_kb = buffer_kb
        self.recv_rate = recv_rate
        self.out = out
        self.detail = detail
        self.occupancy = 0
        self.queue = []          # sizes of received but not processed segments
        self.received = 0
```

```
self.acks_generated = 0
self.full_events = 0
self.buffer_was_full = False
self.occupancy_history = []
self.window_history = []

def record(self):
    # keep a sample of occupancy and advertised window after each change
    self.occupancy_history.append(self.occupancy)
    self.window_history.append(self.buffer_kb - self.occupancy)

def run_receive(self):
    o = self.out
    while True:
        seq, size = yield self.data_ch.get()
        self.queue.append(size)
        self.occupancy += size
        self.received += 1
        self.acks_generated += 1
        self.record()
        if self.detail:
            print(f"Segment S{seq} Received", file=o)
            print(f"Buffer Occupancy : {self.occupancy} KB", file=o)
            print(f"Advertised Window : {self.buffer_kb - self.occupancy} KB", file=o)
            print(f"ACK S{seq} Sent", file=o)
        yield self.ack_ch.put(("ACK", seq, self.buffer_kb - self.occupancy))
        if self.occupancy == self.buffer_kb:
            self.full_events += 1
            self.buffer_was_full = True
            if self.detail:
                print("Receive Buffer Full", file=o)
                print("Sender Transmission Temporarily Restricted", file=o)

def run_process(self):
    o = self.out
    while True:
        if not self.queue:
            # idle poll when nothing is waiting to be processed
            yield self.env.timeout(0.01)
            continue
        size = self.queue[0]
        yield self.env.timeout(size / self.recv_rate)
        self.queue.pop(0)
        self.occupancy -= size
        self.record()
```

```
        if self.detail:
            print("Processing Data...", file=o)
            print(f"Buffer Occupancy : {self.occupancy} KB", file=o)
            print(f"Advertised Window : {self.buffer_kb - self.occupancy} KB", file=o)
        if self.buffer_was_full:
            self.buffer_was_full = False
            if self.detail:
                print("Buffer Space Available", file=o)
                print("Sender Can Continue Transmission", file=o)
        # window update so a waiting sender can resume
        yield self.ack_ch.put(("WND", 0, self.buffer_kb - self.occupancy))

def summary(self):
    o = self.out
    occ = self.occupancy_history
    wnd = self.window_history
    print("Segments Received          :", self.received, file=o)
    print("Segments Processed          :", self.received, file=o)
    print("ACKs Generated                 :", self.acks_generated, file=o)
    if occ:
        print("Average Buffer Occupancy        :", f"{sum(occ) / len(occ):.2f} KB", file=o)
        print("Maximum Buffer Occupancy         :", f"{max(occ)} KB", file=o)
        print("Average Advertised Window       :", f"{sum(wnd) / len(wnd):.2f} KB", file=o)
    print("Full Buffer Events               :", self.full_events, file=o)
    print(file=o)
```

## SIMULATION DRIVER - run\_tcp\_sim.py

```
import os
import simpy

# runs the tcp flow control simulation for the five scenarios.
# tc1 is printed with full per-segment detail, the others as summaries

from tcp_client import TCPClient
from tcp_server import TCPServer

exp_dir = os.path.dirname(os.path.abspath(__file__))
client_out = open(os.path.join(exp_dir, "tcp_client_output.txt"), "w", encoding="utf-8")
server_out = open(os.path.join(exp_dir, "tcp_server_output.txt"), "w", encoding="utf-8")

DELAY = 0.05 # one-way transmission delay in seconds

# total_kb, seg_kb, init window kb, receiver buffer kb, sender rate, receiver rate
scenarios = [
    ("TC1", 10, 1, 4, 8, 2, 4),
    ("TC2", 20, 1, 8, 4, 8, 4),
    ("TC3", 20, 1, 8, 2, 8, 2),
```

```
    ("TC4", 20, 1, 8, 8, 8, 4),
    ("TC5", 20, 2, 8, 8, 8, 4),
]

for tag, total_kb, seg_kb, init_win, buffer_kb, send_rate, recv_rate in scenarios:
    detail = tag == "TC1"
    print(f"===== {tag} =====", file=client_out)
    print(f"===== {tag} =====", file=server_out)
    env = simpy.Environment()
    data_ch = simpy.Store(env, capacity=float("inf"))
    ack_ch = simpy.Store(env, capacity=float("inf"))
    client = TCPClient(env, data_ch, ack_ch, total_kb, seg_kb, init_win, send_rate, DELAY, buffer_kb, client_out, detail)
    server = TCPServer(env, data_ch, ack_ch, buffer_kb, recv_rate, server_out, detail)
    env.process(client.run())
    env.process(server.run_receive())
    env.process(server.run_process())
    # generous bound: generation time plus processing of every segment
    segments = total_kb // seg_kb
    env.run(until=segments * (seg_kb / send_rate) + segments * (seg_kb / recv_rate) + 5.0)
    server.summary()

client_out.close()
server_out.close()
print("Experiment complete. Check tcp_client_output.txt and tcp_server_output.txt.")
```

## CLIENT OUTPUT (FULL TRACE FOR TC1 + SUMMARIES)

```
===== TC1 =====
TCP CLIENT SIMULATION
Total Data      : 10 KB
Segment Size    : 1 KB
Initial Window  : 4 KB
Sender Rate     : 2 KB/s
Transmission Delay: 0.05 s

Segment S1 Transmitted
Segment S2 Transmitted
Segment S3 Transmitted
Segment S4 Transmitted
Sender Waiting - Receiver Window Full
ACK Received : S1
Advertised Window : 7 KB
Segment S5 Transmitted
Sender Waiting - Receiver Window Full
ACK Received : S2
Advertised Window : 7 KB
Segment S6 Transmitted
Sender Waiting - Receiver Window Full
ACK Received : S3
Advertised Window : 7 KB
Segment S7 Transmitted
Sender Waiting - Receiver Window Full
```

```
ACK Received : S4
Advertised Window : 7 KB
Segment S8 Transmitted
Sender Waiting - Receiver Window Full
ACK Received : S5
Advertised Window : 7 KB
Segment S9 Transmitted
Sender Waiting - Receiver Window Full
ACK Received : S6
Advertised Window : 7 KB
Segment S10 Transmitted
ACK Received : S7
Advertised Window : 7 KB
ACK Received : S8
Advertised Window : 7 KB
ACK Received : S9
Advertised Window : 7 KB
ACK Received : S10
Advertised Window : 7 KB
Transmission Completed
Total Segments Sent      : 10
Total ACKs Received      : 10
Total Waiting Events     : 6
Total Transmission Time: 5.50 s
Throughput                : 1.82 KB/s
```

===== TC2 =====

TCP CLIENT SIMULATION

```
Total Data      : 20 KB
Segment Size    : 1 KB
Initial Window  : 8 KB
Sender Rate     : 8 KB/s
Transmission Delay: 0.05 s
```

```
Transmission Completed
Total Segments Sent      : 20
Total ACKs Received      : 20
Total Waiting Events     : 16
Total Transmission Time: 4.23 s
Throughput                : 4.73 KB/s
```

===== TC3 =====

TCP CLIENT SIMULATION

```
Total Data      : 20 KB
Segment Size    : 1 KB
Initial Window  : 8 KB
Sender Rate     : 8 KB/s
Transmission Delay: 0.05 s
```

```
Transmission Completed
Total Segments Sent      : 20
```

```
Total ACKs Received    : 20
Total Waiting Events    : 18
Total Transmission Time: 9.23 s
Throughput              : 2.17 KB/s
```

===== TC4 =====

TCP CLIENT SIMULATION

```
Total Data      : 20 KB
Segment Size     : 1 KB
Initial Window   : 8 KB
Sender Rate      : 8 KB/s
Transmission Delay: 0.05 s
```

Transmission Completed

```
Total Segments Sent    : 20
Total ACKs Received    : 20
Total Waiting Events    : 12
Total Transmission Time: 3.50 s
Throughput             : 5.71 KB/s
```

===== TC5 =====

TCP CLIENT SIMULATION

```
Total Data      : 20 KB
Segment Size     : 2 KB
Initial Window   : 8 KB
Sender Rate      : 8 KB/s
Transmission Delay: 0.05 s
```

Transmission Completed

```
Total Segments Sent    : 10
Total ACKs Received    : 10
Total Waiting Events    : 6
Total Transmission Time: 3.35 s
Throughput             : 5.97 KB/s
```

## SERVER OUTPUT (FULL TRACE FOR TC1 + SUMMARIES)

===== TC1 =====

Segment S1 Received

Buffer Occupancy : 1 KB

Advertised Window : 7 KB

ACK S1 Sent

Processing Data...

Buffer Occupancy : 0 KB

Advertised Window : 8 KB

Segment S2 Received

Buffer Occupancy : 1 KB

Advertised Window : 7 KB

ACK S2 Sent

```
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segment S3 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S3 Sent
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segment S4 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S4 Sent
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segment S5 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S5 Sent
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segment S6 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S6 Sent
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segment S7 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S7 Sent
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segment S8 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S8 Sent
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segment S9 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S9 Sent
Processing Data...
Buffer Occupancy : 0 KB
```

```
Advertised Window : 8 KB
Segment S10 Received
Buffer Occupancy : 1 KB
Advertised Window : 7 KB
ACK S10 Sent
Processing Data...
Buffer Occupancy : 0 KB
Advertised Window : 8 KB
Segments Received      : 10
Segments Processed     : 10
ACKs Generated         : 10
Average Buffer Occupancy : 0.50 KB
Maximum Buffer Occupancy : 1 KB
Average Advertised Window : 7.50 KB
Full Buffer Events      : 0
```

===== TC2 =====

```
Segments Received      : 20
Segments Processed     : 20
ACKs Generated         : 20
Average Buffer Occupancy : 2.90 KB
Maximum Buffer Occupancy : 4 KB
Average Advertised Window : 1.10 KB
Full Buffer Events      : 13
```

===== TC3 =====

```
Segments Received      : 20
Segments Processed     : 20
ACKs Generated         : 20
Average Buffer Occupancy : 1.45 KB
Maximum Buffer Occupancy : 2 KB
Average Advertised Window : 0.55 KB
Full Buffer Events      : 19
```

===== TC4 =====

```
Segments Received      : 20
Segments Processed     : 20
ACKs Generated         : 20
Average Buffer Occupancy : 3.85 KB
Maximum Buffer Occupancy : 7 KB
Average Advertised Window : 4.15 KB
Full Buffer Events      : 0
```

===== TC5 =====

```
Segments Received      : 10
Segments Processed     : 10
ACKs Generated         : 10
Average Buffer Occupancy : 5.00 KB
Maximum Buffer Occupancy : 8 KB
Average Advertised Window : 3.00 KB
Full Buffer Events      : 4
```

## PERFORMANCE COMPARISON

| Test Case | Init Window (KB) | Buffer (KB) | Sender (KB/s) | Receiver (KB/s) | Waiting Events | Time (s) | Throughput (KB/s) | Max Occupancy (KB) | Full Events |
|-----------|------------------|-------------|---------------|-----------------|----------------|----------|-------------------|--------------------|-------------|
| TC1       | 4                | 8           | 2             | 4               | 6              | 5.50     | 1.82              | 1                  | 0           |
| TC2       | 8                | 4           | 8             | 4               | 16             | 4.23     | 4.73              | 4                  | 13          |
| TC3       | 8                | 2           | 8             | 2               | 18             | 9.23     | 2.17              | 2                  | 19          |
| TC4       | 8                | 8           | 8             | 4               | 12             | 3.50     | 5.71              | 7                  | 0           |
| TC5       | 8                | 8           | 8             | 4               | 6              | 3.35     | 5.97              | 8                  | 4           |

## TASKS PERFORMED

| Task                    | Details                                                                                                                               | Result                                                                                        |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Sliding window transmit | Client transmits a segment only when outstanding data is below the effective window (minimum of initial window and advertised window) | No segment was ever sent beyond the receiver's advertised window                              |
| Window updates          | Receiver sends a window update with the new free space after each processing step                                                     | Blocked sender resumed automatically when space became available                              |
| Buffer overflow control | Buffer occupancy tracked for every receive and every processed segment                                                                | Buffer size never exceeded in any scenario, e.g. max 4 KB with a 4 KB buffer in TC2           |
| Ack generation          | Every received segment answered with an ACK carrying the current advertised window                                                    | ACKs generated equalled segments received in all five scenarios                               |
| Scenario comparison     | Five scenarios varying segment size, initial window, buffer size, and sender/receiver rates                                           | TC3 (2 KB buffer) showed 19 full-buffer events while TC4 (8 KB buffer) showed none            |
| Throughput analysis     | Throughput computed as total data divided by total transmission time                                                                  | Small buffer throttled throughput to the receiver rate (2.17 KB/s in TC3 vs 5.97 KB/s in TC5) |

## LEARNING OUTCOMES

- I modelled TCP flow control with a sliding window governed by the receiver's advertised window.
- I implemented window updates so a sender blocked on a full window resumes automatically.
- I observed that a small receive buffer throttles the sender to the receiver's processing rate.

- I computed average and maximum buffer occupancy, waiting events, and throughput from real runs.
- I understood why the advertised window never lets the receive buffer overflow in any scenario.